P7 – Handwritten Sudoku Reader and Solver (NN + Search + CSP)

Project report · Team seed 2024 · Students Name: [Haneem Saber Mousa, 320240154] · [Reem Kareem Mohamed , 320240213].

1. Problem

A neural network reads a 9×9 mosaic of handwritten digits. Sometimes it misreads one digit, which can create two equal digits in one row, column or box, so the puzzle looks broken and cannot be solved. Our system finds the most likely set of misread cells, corrects them, and solves the puzzle. Possible users are puzzle apps and educational tools.

The three parts cooperate in a fixed pipeline: **NN** reads the grid and gives a probability for every digit → **AC-3** detects the conflict → **Search** (UCS / Greedy / A*) tries the cheapest correction sets → **CSP** (Backtracking, Forward Checking, AC-3) tests each corrected puzzle, and the first solvable one wins. If no correction set works, the system answers `UNREADABLE` and lists the least confident cells.

Important: the mosaics are built from MNIST digit images. They are *not* photographs of real Sudoku sheets, and the misreads we study are injected on purpose (Section 2).

2. Data

The full column-by-column description is in `docs/data_dictionary.md`. Summary:

- **Real:** MNIST images and labels (60,000 training and 10,000 test images; training and test writers are different people). A local copy is kept in `data/raw/`; the md5 checksums of the four files are verified by `make_data.py`.
- **Split:** the 60,000 training images are shuffled with seed 0 and 10,000 become validation; then digit 0 is dropped (Sudoku has only 1–9). Result: 45,072 train / 9,005 validation / 9,020 test images. The test set is never used for training or tuning. Pixels are scaled to 0–1.
- **Generated:** Sudoku solution grids (randomised backtracking), puzzles (cells removed one by one while the solution stays unique; 25, 30 or 35 givens), mosaics (each given cell is a random MNIST *test* image of the right digit), and blank cells (faint noise).
- **Blank-cell rule:** a cell is blank if fewer than 10 pixels are brighter than 0.5. The NN is not used for this.

- **Hard-digit injection:** the natural error rate of the NN is only about 2.3%, so there is often nothing to correct. For level *k* we choose *k* given cells and show a test image of the right digit that the calibrated NN misreads while the true digit is still in its top 3. All other cells show images the NN reads correctly. Thus level *k* means exactly *k* misreads. This is an experimental device, not the natural error rate.
- **Seeds:** every team has its own generator seed (`TEAM_SEED = 2024` in `p7sudoku/config.py`); it shifts all random seeds of the generator and the experiments.

3. Neural network

The classifier is a NumPy multilayer perceptron with architecture 784 – 128 – 64 – 9 (ReLU, softmax output, Adam, 12 epochs; the epoch with the best validation accuracy is kept). The baseline is multinomial logistic regression (784 – 9, same training code). Because the probabilities become *costs* of changing a cell, they must be well calibrated, so we apply temperature scaling: one temperature T is chosen on the validation set to minimise the negative log-likelihood (T = 1.387).

Model	Test accuracy	NLL	ECE (15 bins)	Temperature
Logistic regression	0.9218	0.2727	0.0203	1.000
NN (uncalibrated)	0.9772	0.0756	0.0072	1.000
NN + temperature scaling	0.9772	0.0718	0.0036	1.387

The NN is clearly better than the baseline in accuracy (97.7% vs 92.2%) and NLL. Temperature scaling does not change accuracy (it does not change the arg-max) but lowers NLL and halves the expected calibration error (0.0072 → 0.0036). The confusion matrix (`results/classifier.png`) shows that the main confusions are 9→4 (14 cases), 7→2 (13) and 5→3 (11). In the reliability diagram the low-confidence bins contain very few samples, so those points are noisy and should not be over-interpreted. Calibration is measured on MNIST only.

4. Search

Problem formulation.

- *State:* a set of corrections (cell, new digit); each cell is changed at most once; at most 3 corrections.
- *Action:* change one more uncertain cell to one of its top alternatives. Candidates are the 12 least confident given cells, with the top-3 alternative digits each.

- *Cost of one change:* max(0, ln p(read) − ln p(new)). It is never negative. The path cost is the sum over cells, so it does not depend on the order of actions. Example from the project brief: the NN reads a digit with p = 0.52 while the true digit has p = 0.45, so correcting it costs ln(0.52/0.45) = 0.145, which is cheap (this exact case is a unit test).
- *Goal:* the corrected puzzle is solvable by the CSP (optionally: it has exactly one solution). Grids with equal digits in a unit are rejected immediately without running the CSP; this is exactly what AC-3 detects.

Algorithms. UCS orders by path cost g and is optimal. Greedy best-first orders by h = number of remaining duplicate-digit conflicts and is not optimal. A* orders by g + h with the admissible heuristic below.

Admissible heuristic. Every duplicate-digit conflict is a pair of equal cells, and at least one of the two cells must change. We take a set of cell-disjoint conflicts; each of them needs a *different* changed cell, which costs at least the cheaper of its two cells' cheapest alternatives. The sum of these minima is a lower bound on the true remaining cost, so h never overestimates. If the conflicts need more cells than the remaining budget allows, h = ∞ and the state is pruned (it provably cannot be repaired within 3 corrections).

Correctness check. `tests/test_search.py` compares UCS and A* with a brute-force enumeration of all correction sets on small cases: both return the brute-force optimal cost. It also tests the brief's known-answer example, that costs are never negative, that counters are consistent (generated ⩾ expanded), that A* never expands more nodes than UCS on the test case, and that the budget is respected.

*Experiment 1 – UCS vs Greedy vs A.** 100 puzzles per level (5 seeds × 20 puzzles), 30 givens, 0–4 misreads, mean over 5 seeds (full table with standard deviations: `results/e1_search.md`; plot: `results/e1_search.png`). The median is pooled over all puzzles, because a few hard puzzles dominate the mean (large standard deviation).

Misreads	UCS nodes (mean / median)	A* nodes (mean / median)	Greedy nodes (mean / median)	Exact recovery UCS = A*	Exact recovery Greedy	Cost UCS = A*	Cost Greedy
0	0 / 0	0 / 0	0 / 0	1.00	1.00	0.00	0.00
1	133.9 / 1	1.1 / 1	132.4 / 1	0.86 ± 0.065	0.86 ± 0.065	1.22	1.22
2	80.6 / 6	4.8 / 3	68.2 / 2	0.64 ± 0.147	0.58 ± 0.120	2.50	2.74
3	181.5 / 18	7.5 / 5	134.6 / 3	0.43 ± 0.045	0.40 ± 0.061	3.32	3.50
4	3673 / 6571	33.4 / 12.5	3489 / 6571	0.00	0.00	5.49	5.70

Reading the results.

- UCS and A* always return the same cost (both are optimal), but A* expands far fewer nodes: at level 3 the mean is 7.5 vs 181.5, at level 4 it is 33 vs 3673. The heuristic is informative because conflicts directly tell us which cells must change.
- Greedy expands fewer nodes than UCS in the median but returns a more expensive correction at levels 2–4 (2.74 vs 2.50 at level 2) and a lower exact-recovery rate (0.58 vs 0.64). It is fast but not reliable. At level 1 all three algorithms behave the same, because the cheapest candidate is almost always the right one.
- At level 4 no algorithm can recover the true puzzle, because the budget is 3 corrections by design. The median of 6571 nodes is the whole search space: no goal exists inside the budget, so every state is expanded (A* prunes most of it with h = ∞).
- *Exact recovery* is lower than *solvable found* (for example 0.64 vs 0.99 at level 2). A wrong repair can also produce a solvable puzzle; this is the main limit of the "solvable" goal and is discussed in Section 7.

5. CSP

81 variables (cells), domains 1–9, givens fixed, all-different constraints on every row, column and 3×3 box. Backtracking (BT), Forward Checking (FC) and AC-3 are three independent switches, so every combination can be run. Counters: assignments, backtracks, AC-3 revisions and prunings, FC prunings. Ordering is static (row-major) in the experiment so that the effect of each part is visible; MRV ordering is available and used in the generator and the search goal test.

Experiment 2 – BT vs BT+FC vs BT+FC+AC-3 (5 seeds × 20 puzzles per cell, assignment cap 200,000; `results/e2_csp.md`, plot `results/e2_csp.png`). Mean assignments per puzzle:

Givens	BT	BT+FC	BT+FC+AC-3
25	41,287 (solved 0.94)	10,925	5,752
30	5,523	1,193	645
35	499	187	72

FC removes about 4–5× of the assignments and AC-3 about 2× more. AC-3 has its own cost: at 35 givens BT+FC is faster in seconds, even though AC-3 makes fewer assignments. BT at 25 givens is stopped by the cap in 6% of the runs, so its mean is a lower bound.

6. Integration

The flow in `integration.py` is: NN probabilities → blank-cell rule → AC-3 conflict detection → Search over correction sets → CSP goal test → solution. For every solved puzzle a correction log is written (`results/correction_log.csv` : cell, row, column, read digit, new digit, cost). If no correction set within the budget works, the result is `UNREADABLE` with the least confident cells. `python run_demo.py` shows the input mosaic, the search trace, the CSP pruning table (BT, BT+FC, BT+AC-3, BT+FC+AC-3), the system with correction on versus off, and the figure `results/demo.png` .

Experiment 3 – NN-guided correction vs no correction vs random correction (30 givens, exact-solution recovery; `results/e3_correction.md` , plot `results/e3_correction.png`):

Method	1 misread	2 misreads	3 misreads
NN-guided (UCS)	0.84	0.66	0.49
NN-guided (UCS, unique-solution goal)	0.94	0.88	0.86
No correction	0.00	0.00	0.00
Random correction (200 tries)	0.16	0.00	0.00

Without correction the system is `UNREADABLE` in 89–100% of the puzzles or, rarely, solves a puzzle with a wrong repair. Random correction almost never finds the true digits. NN-guided correction wins clearly, which shows that the calibrated probabilities carry real information. The *unique-solution goal* raises exact recovery to 0.94 / 0.88 / 0.86, because a wrong repair rarely gives a puzzle with exactly one solution. The same goal repeated inside Experiment 1 gives 0.96 / 0.92 / 0.75 (`results/e1_search_unique.md`). Differences between Experiments 1 and 3 for the same method (for example 0.86 vs 0.84 at level 1, or 0.75 vs 0.86 at level 3 with the unique goal) come from the different puzzle seeds of the two experiments (1000+s and 9000+s), i.e. sampling noise on 100 puzzles, not from a different algorithm.

7. Limits

1. **"Solvable" is a weak goal.** A wrong repair can also give a solvable puzzle (`solved_but_wrong` 0.12 / 0.35 / 0.55 for 1–3 misreads in Experiment 1). The unique-solution goal and the `max_cost` limit help, but neither is perfect. We only know that a repair is wrong because we generated the puzzle and know the true answer.
2. **Candidate limits.** The true digit must be among the top-3 alternatives, and the misread cell among the 12 least confident cells; otherwise it cannot be fixed.

3. **Budget.** With at most 3 corrections, level-4 puzzles cannot be recovered completely.
4. **Synthetic data.** The mosaics come from MNIST, not from photographs of real Sudoku, and the injected misreads are artificial. MNIST digits are centred and clean; real handwriting in grid cells is harder.
5. **Calibration** is measured on MNIST only, and the low-confidence bins of the reliability diagram contain few samples.
6. **Speed.** The code is pure Python and NumPy, which limits the size of the experiments (100 puzzles per level, 5 seeds).

8. Conclusions

Calibrated NN probabilities turn a misread digit into a cost. With that cost, UCS and A* find the cheapest correction – A* with far less search than UCS (7.5 vs 181.5 nodes at three misreads) and with the same optimal cost – while Greedy is cheaper per step but gives worse corrections. Forward Checking and AC-3 reduce the CSP work by roughly 4–5× and a further 2×. The whole system recovers the true puzzle in 84% / 66% / 49% of cases for 1 / 2 / 3 misreads (94% / 88% / 86% with the unique-solution goal), where no correction and random correction fail. The main remaining weakness is the weak "solvable" goal, which the unique-solution goal reduces but does not remove.

Appendix

- **How to run:** see `README.md` (setup with pinned `requirements.txt`, then `make_data.py`, `train_nn.py`, `python -m unittest discover -s tests -t . -v` (33 tests), `run_demo.py --seed 0`, `run_experiments.py --unique-e1`).
- **Tests:** 5 normal, 3 edge and 2 impossible system cases (impossible cases print a clear `UNREADABLE` message), plus unit tests for the CSP, search (against brute force), NN and generator.
- **Git log:** Git Repository & Collaboration Log:
  "Development was primarily conducted in a shared local Visual Studio Code environment to ensure seamless integration between the CSP solver, Neural Network pipeline, and experiment benchmarks. Once code execution and testing were fully validated, the project was committed and pushed to GitHub under a single unified commit (0ed18c6: Initial commit: Sudoku project code and results). Subsequent documentation updates were tracked directly on the remote repository."